

Comparative Study Of Various Approaches Of Dijkstra Algorithm

1st Divyam Verma

Department of Informatics,
(School of Computer Science)
University of Petroleum and Energy
Studies
Dehradun, Uttarakhand, India
divyam0216@gmail.com

2nd Devansh Messon

Department of Informatics,
(School of Computer Science)
University of Petroleum and Energy
Studies
Dehradun, Uttarakhand, India
devansh.messon28@gmail.com

3rd Mayank Rastogi

Department of Informatics,
(School of Computer Science)
University of Petroleum and Energy
Studies
Dehradun, Uttarakhand, India
Mayikmj@gmail.com

4th Amit Singh*(corresponding author)

Department of Informatics,
(School of Computer Science)
University of Petroleum and Energy
Studies
Dehradun, Uttarakhand, India
amit.singh1@ddn.upes.ac.in

Abstract—Dijkstra algorithm is not improved in terms of space and time-complexity, the algorithm is muddled to apply in network analysis for enormous test cases. To handle this huge time complexity, advanced structures are applied to enhance Dijkstra algorithm. The use of a keen and numerical procedure speeds up the enhanced Dijkstra by multiple times. In this paper four approaches using advanced data structures like linked list, Fibonacci heap, binary heap and bi-directional Dijkstra are applied and their performance is also analyzed and compared on two classes of graphs, dense and sparse graphs.

Keywords—Dijkstra algorithm, graphs, binary heap, Fibonacci heap, Linked List

I. INTRODUCTION

With the progression of innovation and the broad exchange of information, the Dijkstra algorithm is applied to some real-world problems like digital mapping services, social networking applications, telephone and road networking, flighting agenda, designating file server, robotic path, financial market, business, and other network-oriented problems [1]. As the network is developing and growing step by step, the multifaceted nature of such issues is expanding complex. Subsequently, best fitting algorithms and their methodologies become significant. The current paper focuses around execution time analysis of different implementations of the Dijkstra algorithm for dense and sparse graphs.

A. Dijkstra algorithm

The shortest path algorithm is a traditional algorithm in graph theory with the purpose to locate the shortest path from a specific source to various destinations. Dijkstra is a solitary source shortest path algorithm and the goal is to improve the execution time as well as memory utilization of the algorithm. Dijkstra algorithm does not have negative edge weights. It applies a greedy strategy to ideally tackle single-source shortest path problem.

The algorithm picks up a vertex with the minimum distance from the source, and this picked vertex relaxes up its adjacent vertices and these two stages run $|V|$ times, where V is the number of vertices in the graph. Brute-force based solution of Dijkstra algorithm takes $O(V^2)$ time and $O(V^2)$ space [2].

B. Dense and Sparse Graphs

Regarding the density of a graph, there are two classes, sparse and dense graphs [3].

Dense graph : Dense graph is a graph where E the number of edges is which near to the maximal number of edges.

The mathematical representation of dense graph is:

Let $G(V, E)$ is a graph where V and E are vertices and edges respectively then dense graph is: $|E| = \theta(|V|^2)$.

Sparse graph: Sparse graph is a graph in which the quantity of edges is near the insignificant number of edges.

The mathematical representation of dense graph is:

Let $G(V, E)$ is a graph where V and E are vertices and edges respectively then sparse graph is: $|E| = O(|V|)$.

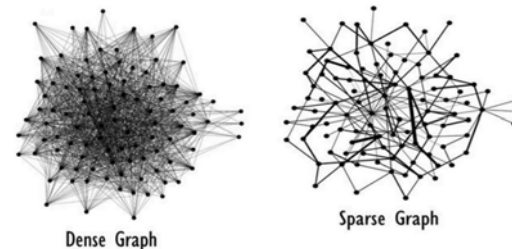


Fig 1. Various types of graphs on the basis of graph density

C. The approaches to optimize the Dijkstra algorithm

Dijkstra algorithm isn't improved in terms of space and time complexity, the algorithm is muddled to apply in network analysis for enormous test cases. For example, in the event that $V=10^{18}$ (number of vertices), at that point finding a minimum element by brute-force based solution will take 10^{18} iterations.

To handle this huge time complexity, advanced structures are applied to enhance Dijkstra algorithm. The use of a keen and numerical procedure speeds up the enhanced Dijkstra by multiple times. Here are some advanced data-structures applied in this analysis.

Linked list : A linked list is used to store the graph proficiently. A linked list is more proficient than a matrix as far as storing the graph. For example, on the off chance that $V=10$ and $E=5$ (sparse graph), at that point the linked list will take 5 units of space, yet the matrix will take 100 units of space. A linked list is the execution of a modern adjacency list

which helps in decreasing the space. Structures and pointers are used to connect heterogenous information related with graph. Linked list is the implementation of modern adjacency list which helps in reducing the space from $O(|V|^2)$ to $O(|E|)$ where E is the number of edges.

Fibonacci heap : A Fibonacci heap is a particular usage of the heap data structure that uses Fibonacci numbers. Fibonacci heap is utilized to actualize the need line component in Dijkstra algorithm, giving the calculation an exceptionally effective running time.

The Fibonacci heap is likewise used to execute a priority queue that finds and deletes the minimum component in $O(\log n)$ iterations(amortized), where n is the number of components in the heap. Reduction key activity is done in amortized cost of $O(1)$. Although Fibonacci loads have gained notoriety for being delayed by and by [4].

Binary heap : A binary heap is the execution of a modern priority queue that proficiently finds and erases the minimum element in $O(\log n)$ iterations, where n is the number of elements in the heap. Relaxation of a vertex additionally takes $O(\log n)$ iterations [5].

In the event that $V=10^{18}$ (number of vertices), at that point finding a minimum element by brute-force based solution will take 10^{18} cycles, and then again, the binary heap will take only 60 iterations to discover the minimum element. Binary heap and linked list aggregately diminished the running time from $O(|V|^2)$ to $O(|E| \log |V|)$.

Bidirectional Dijkstra : Bi-directional method runs the Dijkstra algorithm from the source vertex and objective vertex at the same time and the algorithm ends when a vertex is handled by forward binary heap and backward binary heap [6]. In the event that the graph structure is straight and on the off chance that $V=10^5$, at that point the unidirectional Dijkstra calculation will take 10^5 cycles to get ended yet then again, the bidirectional Dijkstra calculation will take $5*10^4$ iterations to end, which is improving the running time multiple times quicker. The primary component of this method is the amazing decrease in the quantity of cycles/iterations and the simplicity in finding the shortest path/route from one source to destination.

II. EXPERIMENTAL STUDY

In the present paper, all four approaches were implemented in ANSI C programing and executed on GCC compiler. The flowchart, algorithm and code snippets are mentioned.

A. Brute-force method:

1. Distance of nodes(except source node) from source node is infinity.
2. Find the unvisited node with minimum distance //complexity in $O(n)$.
3. Mark the picked node as visited.
4. Relax the nodes adjacent to picked node.
5. Adjacent nodes which are marked as visited cannot be relaxed further.
6. Relaxing condition:
7. $\text{if}(\text{dist}[\text{picked vertex}] + \text{edge}(\text{picked vertex}, \text{adjacent vertex}) < \text{dist}[\text{adjacent vertex}])$

8. Repeat the above steps n times or break if there is no vertex found with minimum distance.

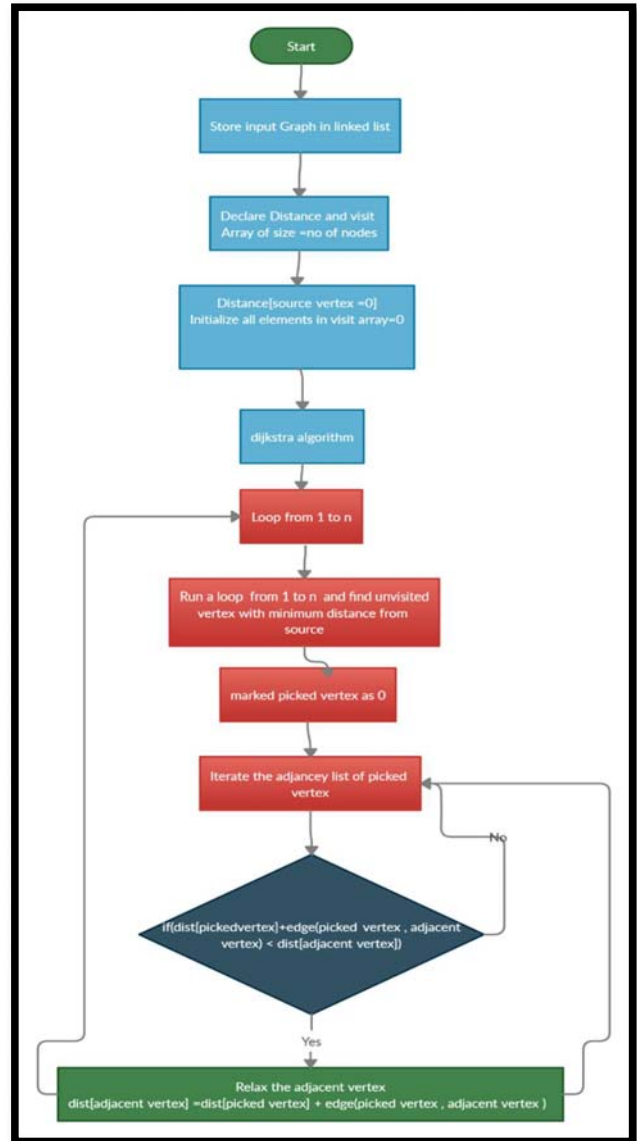


Fig 2: Flowchart of Brute-force method

```

int dijkstra_bruteforce(int *adjac, int *dist, int *previous, int *visit)
{
    int till=nodes;
    while(till>0)
    {
        int min=INT_MAX, pick=0;
        for(int i=1; i<=nodes; i++)
        {
            if(min>dist[i] && visit[i]==0) { min=dist[i]; pick=i; ok=1; } //Picking out minimum vertex
        }
        if(ok==0) break;
        visit[pick]=1; //marking as visited
        //traverse nodes adjacent to picked vertex
        struct graph *pick;
        ok=0; adjacout=edge;
        struct adjacout {int *adj; int *dist; int *previous; int *visit;};
        struct adjacout *adjacout;
        adjacout=ptr=adjac[pick];
        while(ptr->next!=0)
        {
            if(visit[ptr->adj]==0 && dist[pick]+edge==dist[ptr->adj] && dist[pick]!=INT_MAX && edge!=0)
            {
                dist[ptr->adj]=dist[pick]+edge; //relaxation of nodes adjacent to picked vertex
                previous[ptr->adj]=pick;
            }
            ptr=ptr->next;
        }
        till--;
    }
}
  
```

Fig 3. Implementation of Brute-force method

B. Fibonacci Heap Implementation

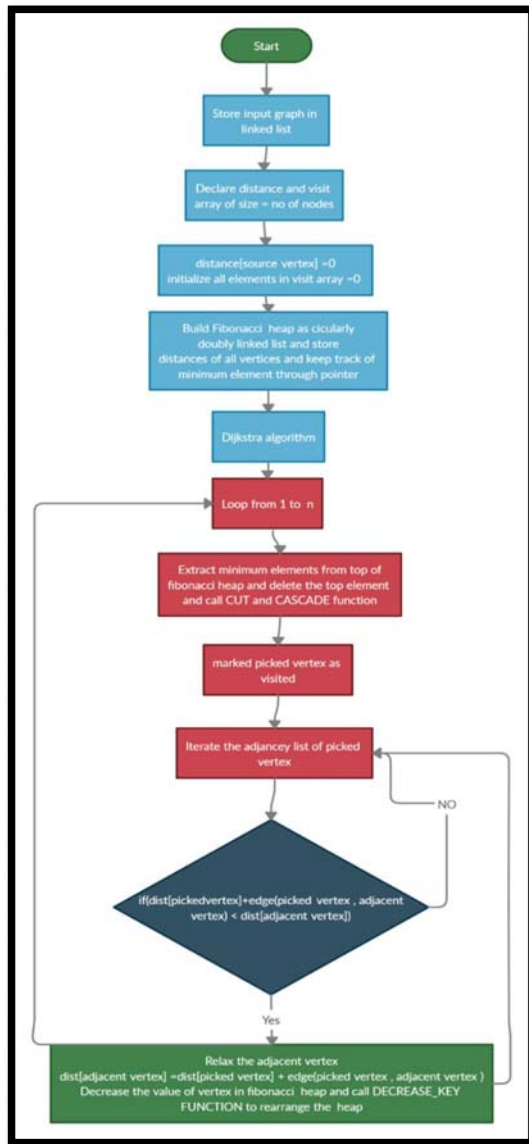


Fig 4. Flowchart of Fibonacci heap implementation

Sub-algorithm Building fibonacci heap:

1. Allocate a new node.
2. Make its parent, child, left, right as NULL.
3. degree, mark = 0
4. Distance of nodes from source node is infinity except source node.
5. If heapsize == 0:
6. Minimum element = input
7. if heapsize > 0:
8. Minimum element = min(Minimum element, input)

Sub-algorithm Deleting Minimum element from Fibonacci heap :

1. Unlink the minimum node from the root list and add all its children to the root list.
2. Degree of node = number of children.
3. All Subtrees should have unique degree.
4. Max. Degree in the Root list = $\log_2(N)$

5. Declare array pointer which is pointing to node which has degree = array's index
6. Traverse the root list of Fibonacci heap.
7. If array[current node's degree] == NULL:
8. then array[current node's degree] will point to current node.
9. If array[current node's degree] != NULL:
10. If array[current node's degree] value > current node value
11. then make current node as parent of array [current node's degree]
12. array[current node's degree] will point to current node
13. else vice versa
14. After every subtree has unique degree in the Fibonacci heap.
15. Traverse the array pointer and find the minimum element of the Fibonacci heap

```

int build_fibonacciheap(int i, int disti)
{
    ptr = (struct fibo*) malloc(sizeof(struct fibo));
    ptr->parent = NULL;
    ptr->child = NULL;
    ptr->left = ptr;
    ptr->right = ptr;
    ptr->key = disti;
    ptr->index = i;
    ptr->mark = 0;
    ptr->degree = 0;
    map[i] = ptr;

    if (min == NULL) min = ptr;
    else
    {
        (min->left)->right = ptr;
        ptr->left = min->left;
        ptr->right = min;
        min->left = ptr;
        if (ptr->key < min->key) min = ptr;
    }
}

```

Fig 5. Implementation of building of Fibonacci heap

```

int extract_min()
{
    temp = min;
    if (temp->child == NULL)
    {
        traverse = temp->child;
        do
        {
            ptr = traverse->right;
            (min->left)->right = traverse;
            traverse->left = min->left;
            traverse->right = min;
            min->left = traverse;
            if (traverse->key < min->key) min = traverse;
            traverse->parent = NULL;
            traverse = ptr;
        } while (ptr != temp->child);

        (temp->left)->right = temp->right;
        (temp->right)->left = temp->left;
        min = min->right;
        consolidate();
        n--;
        return min->key;
    }

    int consolidate()
    {
        maxdeg = log(nodes)/log(2);
        struct fibo* arr[maxdeg+1];
        for (int i = 0; i < maxdeg; i++) arr[i] = NULL;
        for (int i = 0; i < nodes; i++)
        {
            if (!visit[i])
            {
                min = min;
                struct fibo* temp;
                if (deg == 0)
                {
                    (arr[deg] = min) = min;
                    min = min->right;
                }
                else
                {
                    (arr[deg] = min) = min;
                    min = min->right;
                }
                temp = min->child;
                while (temp != NULL)
                {
                    (make_child->left) = min->left;
                    (make_child->right) = min->right;
                    (make_child->parent) = min;
                    min = temp;
                }
                (make_child->left) = min->left;
                (make_child->right) = min->right;
                (make_child->parent) = min;
                min = make_child;
            }
        }
    }
}

```

Fig 6. Implementation of deleting min element from Fibonacci heap

Algorithm Decreasing key of Fibonacci heap:

1. Map array is used to get the position of a particular element in heap.
2. Assign: value of ith element = Decreased value.

Sub-Algorithm CUT Function():

1. If value of parent of ith node > value of ith node:
2. then : Move ith node to the root list and mark it 0.
 - i. Degree of parent is reduced by 1

Sub-Algorithm CASCADE Function():

1. If parent of i th node is marked as 0:
2. then parent of i th node is marked as 1
3. else if parent of i th node is already marked as 1.
4. then: CUT()
CASCADE_CUT()

```

void cut(struct fibo* found, struct fibo* temp)
{
    if(found==found->right)temp->child=NULL;
    if(temp->child==found && temp->child!=NULL)
    {
        temp->child=found->right;
    }
    (found->left)->right=found->right;
    (found->right)->left=found->left;
    found->right=found;
    found->left=found;
    found->parent=NULL;
    (min->left)->right=found;
    found->right=min;
    found->left=min->left;
    min->left=found;
    found->mark=0;
    (temp->degree)--;
}

void cascade_cut(struct fibo* temp)
{
    traverse=temp->parent;
    if(traverse!=NULL)
    {
        if(traverse->mark==0)traverse->mark=1;
        else
        {
            cut(temp, traverse);
            cascade_cut(traverse);
        }
    }
}

```

Fig 7. Implementation of decreasing key Fibonacci heap

C. Binary Heap Implementation**Sub-algorithm Building Minimum Heap:**

1. Distance of nodes from source node is infinity except source node.
2. Put distance[source node]=0 as root of the heap.
3. Traverse the heap array from 2nd index and put distance of all other nodes.

Sub-algorithm Deleting Minimum Node from binary heap:

1. Move the last element of heap to 1st index of heap array.
2. Rearrange the heap such that min. element is on top.
3. Start from top element:
4. Child1=2*(parent's index)+1
5. Child2=2*(parent's index)+2
6. If distances[children] < distance[parent]:
7. Take min. of distances of Child1, Child2 and swap the minimum one with the parent node.
8. Make the minimum element as parent and Repeat the above steps.
9. Else: Do nothing

Sub-algorithm Decrease a value of i th element in heap:

1. Map array is used to get the position of a particular element in heap.
2. Assign value of i th element=Decreased value
3. Heap might get disturbed so rearrange it.
4. Start from the i th element and go upwards in heap. Compare its value with parent.
5. Parent's index in heap= $i/2$;
6. while Parent's value> i th element's value:
7. -swap(parent, i th element)
8. -update the map array
9. - i th element=parent(going upwards).

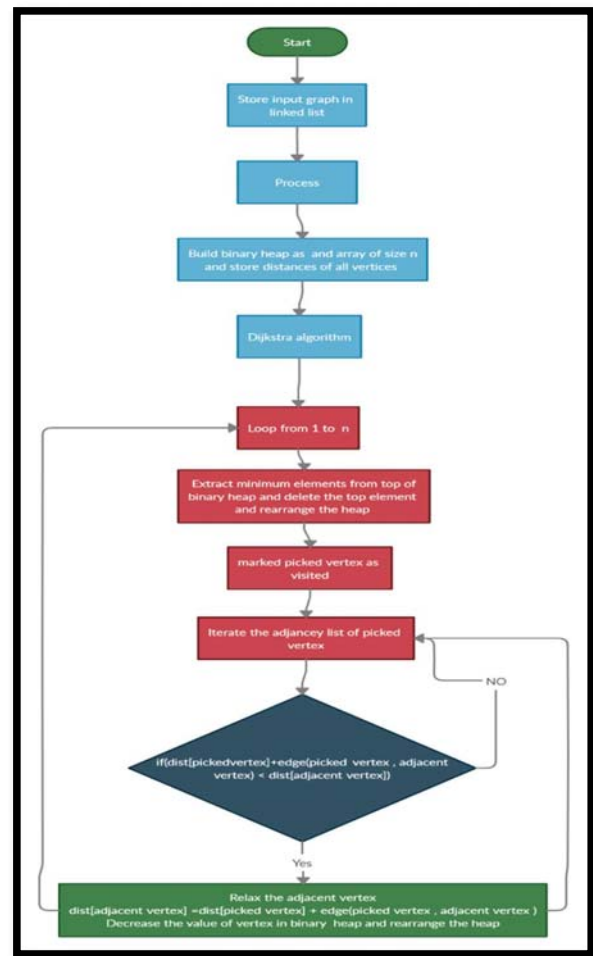


Fig 8. Flowchart of binary heap implementation

```

int build_minheap(int *dist, int *previous, int *map)
{
    cnt=1;
    heap[1].mindist=dist[source];
    heap[1].vertex=source;
    map[heap[1].vertex]=1;
    cnt++;
    for(int i=1; i<=nodes; i++)
    {
        if(i==source){previous[i]=1; continue;}
        else if(i!=source)previous[i]=-1;
        heap[cnt].mindist=dist[i];
        heap[cnt].vertex=i;
        map[heap[cnt].vertex]=cnt;
        cnt++;
    }
}

```

Fig 9. Implementation of building of binary heap

```

int delete_min(int map[])
{
    map[heap[heapsize].vertex]=map[heap[1].vertex];
    heap[1].mindist=heap[heapsize].mindist;
    heap[1].vertex=heap[heapsize].vertex;
    heapsize--;
    /*Compare the first node with its child nodes*/
    cur=1;
    while(1)
    {
        c1=2*cur; //left child
        c2=2*cur+1; //right child
        small=LONG_MAX;
        if(c1<=heapsize && heap[c1].mindist<heap[cur].mindist)small=c1;
        if(c2<=heapsize && heap[c2].mindist<heap[cur].mindist)
        {
            if(small==LONG_MAX){if(heap[c2].mindist<heap[small].mindist)small=c2;}
            else if(small==LONG_MAX)small=c2;
        }
        if(small==LONG_MAX)break;
        swap(&map[heap[small].vertex], &map[heap[cur].vertex]);
        swap(&heap[small].mindist, &heap[cur].mindist);
        swap(&heap[small].vertex, &heap[cur].vertex);
        cur=small;
    }
}

```

Fig 10. Implementation of deleting min element from binary heap

D. Bi-directional Dijkstra Implementation

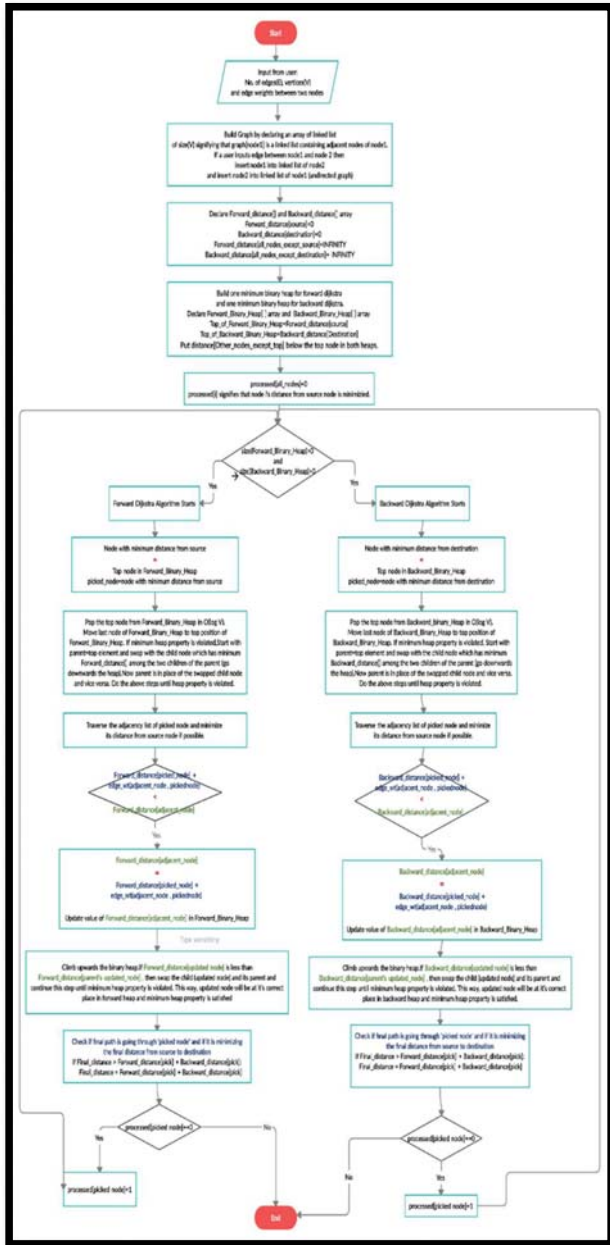


Fig 11. Flowchart of Bi-directional Dijkstra

Optimized_Dijkstra()

1. input(number_of_nodes)
2. input(number_of_edges)
3. FOR LOOP till number_of_edges:
4. input(node_1)
5. input(node_2)
6. input(edge_weight)
7. Call Build_Graph(node_1,node_2,edge_weight)
8. Call Build_Graph(node_2,node_1,edge_weight)
9. declare Forward_distance [] and Backward_distance [] array
10. Forward_distance[source]=0
11. Backward_distance [destination]=0

12. Forward_distance [all_nodes_except_source] = INFINITY
13. Backward_distance [all_nodes_except_destination] = INFINITY

Sub Algorithm Build_Graph (node_1, node_2 , edge_weight)

1. put node_2 into linked_list of node_1
2. put node_1 into linked_list of node_2

Sub Algorithm Build_Binary_Heap (Binary_Heap_type, top)

1. //Binary_Heap_Type means Forward_heap OR Backward_heap
2. declare Binary_Heap_type[] array
3. Top_of_binary_heap=distance[top]
4. Put distance [Other_nodes_except_top] below the top node element

Algorithm Bidirectional_Dijkstra()

1. processed[all_nodes]=0
2. //processed[i] signifies that node i's distance from source node is minimized.
3. WHILE Forward_Heap_size>0 AND Backward_Heap_size>0 :
4. //Forward Dijkstra Starts
5. Dijkstra(Forward_Dijkstra)
6. //Check if final path is going through 'pick' node and if it is minimizing
7. //the final distance from source to destination
8. if Final_distance > Forward_distance[pick] + Backward_distance[pick]:
9. Final_distance = Forward_distance[pick] + Backward_distance[pick]
10. //Above line is minimizing the final distance from source to destination
11. if processed[pick]==1:break
12. //if pick's distance is already minimized then Bidirectional algorithm terminates.
13. else processed[pick]=1
14. //pick's distance from source node is minimized.
15. //Backward Dijkstra Starts
16. Dijkstra(Backward_Dijkstra)
17. //Check if final path is going through 'pick' node and if it is minimizing
18. //the final distance from source to destination
19. if Final_distance > Forward_distance[pick] + Backward_distance[pick]:
20. Final_distance = Forward_distance[pick] + Backward_distance[pick]
21. //Above line is minimizing the final distance from source to destination
22. if processed[pick]==1: break
23. //if pick's distance is already minimized then Bidirectional algorithm terminates.
24. else processed[pick]=1
25. //pick's distance from source node is minimized.

SubAlgorithm Dijkstra(Dijkstra_type)

1. //Binary_heap_type means Forward_Dijkstra OR Backward_Dijkstra
2. int pick=top_element_of_heap
3. //pick signifies the node with minimum distance from source node.
4. //Delete Top element from Forward_Binary_Heap to access next node in next iteration
5. Delete_Top_Element(Binary_Heap_Type)
//Binary_Heap_Type means Forward_heap OR Backward_heap
6. //Traverse the adjacency_list of pick to relax the adjacent nodes of pick
7. FOR LOOP till size_of_adjacencylist_of_pick:
8. //Relax the adjacent node if possible
9. if
distance_type[pick]+edge_weight(pick,adjacent_node)
< distance_type[adjacent]:
10. //distance_type means Forward_distance OR backward_distance
11. //Relax the adjacent node as condition is satisfied
12. distance_type[adjacent_node]=distance_type[pick]+edge_weight(pick,adjacent_node)
13. Relax_Value_Of_Node(adjacent_node,Binary_Heap_Type)
14. //Reflect the changes in minimum heap

Sub Algorithm Delete_Top_Element (Binary_Heap_Type)

1. //Binary_Heap_Type means Forward_heap OR Backward_heap
2. MOVE last_node_in_heap TO Top_position_in_heap
3. //Top element deleted
4. //Rearrange the heap
5. WHILE True:
6. LEFT_CHILD=2*(Index_of_parent_in_heap)
//left child node
7. RIGHT_CHILD=2*(Index_of_parent_in_heap)+1
//right child node
8. //Go downwards in heap if minimum heap property is violated
9. if Parent > LEFT_CHILD OR RIGHT_CHILD:
10. SWAP Parent and Minimum(LEFT_CHILD,RIGHT_CHILD)
11. Parent=Minimum(LEFT_CHILD,RIGHT_CHILD)
12. else break

```

pick=heap1[1].vertex;
extract_min_F();

ptr=graph[pick];
ok=0,adjacent,edge;
stop=adjac[pick];
while(stop>0 && forwarddist[pick]!=LLONG_MAX)
{
    adjacent=ptr->nodeindex;
    edge=ptr->weight;
    if(map1[adjacent]<=heapsize1 && forwarddist[pick]+edge<forwarddist[adjacent]
        && forwarddist[pick]!=LLONG_MAX && edge>=0)
    {
        forwarddist[adjacent]=forwarddist[pick]+edge;//relaxation of nodes
        //adjacent to picked vertex
        previous[adjacent]=pick;
        update=map1[adjacent];
        heap1[update].mindist=forwarddist[pick]+edge;
        decrease_key_F();
    }
    ptr=ptr->adj;
    stop--;
}
if(dist>forwarddist[pick]+backwarddist[pick])
{
    dist=forwarddist[pick]+backwarddist[pick];
    common=pick;
}

if(visit[pick]==1){break;}
visit[pick]=1;

```

Fig 12. Implementation of Forward Dijkstra

```

pick=heap2[1].vertex;
extract_min_B();

ptr=graph[pick];
ok=0,adjacent,edge;
stop=adjac[pick];
while(stop>0 && backwarddist[pick]!=LLONG_MAX)
{
    adjacent=ptr->nodeindex;
    edge=ptr->weight;
    if(map2[adjacent]<=heapsize2 && backwarddist[pick]+edge<backwarddist[adjacent]
        && backwarddist[pick]!=LLONG_MAX && edge>=0)
    {
        backwarddist[adjacent]=backwarddist[pick]+edge;//relaxation of nodes
        after[adjacent]=pick;
        update=map2[adjacent];
        heap2[update].mindist=backwarddist[pick]+edge;
        decrease_key_B();
    }
    ptr=ptr->adj;
    stop--;
}
if(dist>forwarddist[pick]+backwarddist[pick])
{
    dist=forwarddist[pick]+backwarddist[pick];
    common=pick;
}

if(visit[pick]==1){break;}
visit[pick]=1;

if(heapsize1==0 || heapsize2==0){break;}

```

Fig13. Implementation of Backward Dijkstra

III. RESULT ANALYSIS AND DISCUSSION

The outcome of execution time (in milliseconds) was done on two distinct classes of the graphs on the basis of graph density (dense and sparse) as shown in the Table I. There are 18 test cases in the experiment, which are used for all four proposed implementations (Brute-force based solution: BFB, Fibonacci heap implementation: FHI, Binary heap implementation: BHI and Bi-Directional Dijkstra: BDD) on same processor (*i5 7th generation, 8GB RAM*). In these test cases a graph is represented by $N*V$ where N is number of nodes and V is number of vertices. Decision point is best performance implementation on the basis of execution time. Yellow coloured rows represent sparse graph test cases.

TABLE I EXECUTION TIME (ms)

$N*V$	Case No	BFB	BHI	BDD	FHI	D-point
1E4 * 1E5	Case 1	392	8	1	282	BDD
1E4 * 5E5	Case 2	485	32	2	291	BDD
1E4 * 10E5	Case 3	526	80	3	303	BDD
1E4 * 500	Case 4	16	1	1	292	BDD/BHI
1E4 * 2000	Case 5	79	1	1	294	BDD/BHI
1E4 * 4000	Case 6	156	2	2	296	BDD/BHI
5E4 * 1E5	Case 7	9417	22	1	9647	BDD
5E4 * 5E5	Case 8	10442	44	4	9664	BDD
5E4 * 10E5	Case 9	10828	66	4	9723	BDD
5E4 * 500	Case 10	125	2	3	9682	BHI
5E4 * 2000	Case 11	437	3	4	9643	BHI
5E4 * 4000	Case 12	806	4	3	9682	BDD
1E5 * 1E5	Case 13	38559	34	2	7023	BDD
1E5 * 5E5	Case 14	39672	68	2	7054	BDD
1E5 * 10E5	Case 15	37172	94	3	7103	BDD
1E5 * 500	Case 16	187	4	1	7162	BDD
1E5 * 2000	Case 17	798	7	3	7191	BDD
1E5 * 4000	Case 18	1515	8	3	7215	BDD

TABLE II RANKING OF IMPLEMENTATION

Case No	BFB	BHI	BDD	FHI
Case 1	4	2	1	3
Case 2	4	2	1	3
Case 3	4	2	1	3
Case 4	3	1	1	4
Case 5	3	1	1	4
Case 6	3	1	1	4
Case 7	3	2	1	4
Case 8	4	2	1	3
Case 9	4	2	1	3
Case 10	3	1	2	4
Case 11	3	1	2	4
Case 12	3	2	1	4
Case 13	4	2	1	3
Case 14	4	2	1	3
Case 15	4	2	1	3
Case 16	3	2	1	4
Case 17	3	2	1	4
Case 18	3	2	1	4

Table II is a continuation of Table I where it represents ranking of all four proposed implementations according to the best to worst execution time for each case (*Case 1-18*). Numbering from 1-4 have following meaning in Table II -

- 1 - Best in terms of execution time
- 2 - Second best in terms of execution time
- 3 - Third best in terms of execution time
- 4 - Forth best in terms of execution time

Based on Table I and II the performance is examined as follows -

A. Analysis 1 (Performance Comparison between BFB and FHI for Sparse graph)

Table III represents an execution time analysis between BFB and FHI for sparse graph test cases. As per the outcomes, BFB performs in a better way than FHI for sparse graphs which can also be concluded from Figure 14. As appeared in the line graphs of Figure 14, BFB is continually increasing in normal pattern as the number of nodes and vertices increment all the while. While on account of FHI, as the number of nodes increment from 1E5 to 5E5 the values and line graph shoots up. Yet, as the diagram turned a little dense, the execution time of FHI got slightly diminished.

TABLE-III EXECUTION TIME OF BFB AND FHI FOR SPARSE GRAPH

$N*V$	BFB	FHI
1E4 * 500	16	292
1E4 * 2000	79	294
1E4 * 4000	156	296
5E4 * 500	125	9682
5E4 * 2000	437	9643
5E4 * 4000	806	9682
1E5 * 500	187	7162
1E5 * 2000	798	7191
1E5 * 4000	1515	7215

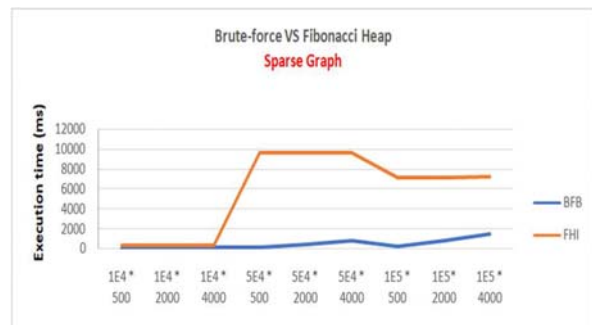


Fig. 14. Performance curve between BFB and FHI for Sparse Graph

B. Analysis 2 (Performance Comparison between BFB and FHI for Dense graph)

Table IV shows to an execution time analysis between BFB and FHI for dense graph test cases. As per the outcomes, FHI

performs far better than BFB for dense graphs. As indicated by Table I and II, there is just one case (*Case 7*) in which BFB has demonstrated better execution time when contrasted with FHI by the distinction of 230ms.

Figure 15, FHI's executing time is continually increasing consistently as dense graph is getting expanded or getting complexed. While on account of BFB, as the number of nodes increase, the line graph shoots-up. In the case of FHI, there is not much difference between the time complexities of sparse graph test cases and dense graphs test cases. According to analysis 1, BFB shows a constant increment in execution time but in the dense graph BFB has shown an abnormal increment in execution time as the number of nodes and vertices increases.

TABLE-IV EXECUTION TIME OF BFB AND FHI FOR DENSE GRAPH

$N*V$	BFB	FHI
1E4 * 1E5	392	282
1E4 * 5E5	485	291
1E4 * 10E5	526	303
5E4 * 1E5	9417	9647
5E4 * 5E5	10442	9664
5E4 * 10E5	10828	9723
1E5 * 1E5	38559	7023
1E5 * 5E5	39672	7054
1E5 * 10E5	37172	7103

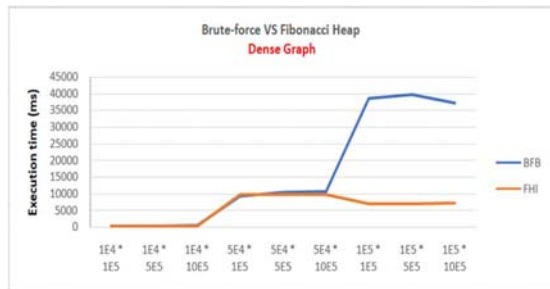


Fig. 15. Performance curve between BFB and FHI for dense Graph

C. Analysis 3 (Performance Comparison between BHI and BDD for Sparse graph)

Table V and figure 3 represent execution time analysis among BHI and BDD for sparse graph test cases. As per the outcomes, both BDD and BHI have similar execution time with some minute differences in some cases. According to Table II, in case 4, 5 and 6 both BHI and BDD have same execution time. In case 10 and 11 BHI has demonstrated better execution time when contrasted with BDD by the distinction of 1ms.

TABLE-V EXECUTION TIME OF BHI AND BDD FOR SPARSE GRAPH

$N*V$	BHI	BDD
1E4 * 500	1	1
1E4 * 2000	1	1
1E4 * 4000	2	2
5E4 * 500	2	3
5E4 * 2000	3	4
5E4 * 4000	4	3
1E5 * 500	4	1
1E5 * 2000	7	3
1E5 * 4000	8	3

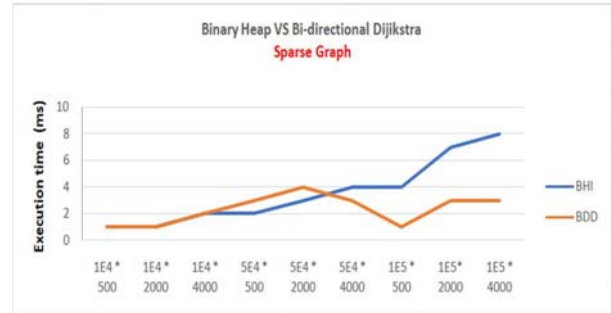


Fig. 16. Performance curve between BHI and BDD for Sparse Graph

D. Analysis 4 (Performance Comparison between BHI and BDD for dense graph)

Table V represents execution time analysis among BHI and BDD for dense graph test cases. As per the outcomes, BDD performs in a way that is better than BHI for dense graphs which can be concluded from Table II and Figure 17. According to Table V The execution time of BDD lies between 1-4 while in the BHI the execution time is in double digit with decreasing-increasing variations, as the number of nodes and vertices increases. These variations can be seen in the Figure 16 as a Saw tooth pattern. The line graph of BDD is practically consistent with a slight aggravation pattern.

TABLE-V EXECUTION TIME OF BHI AND BDD FOR DENSE GRAPH

$N*V$	BHI	BDD
1E4 * 1E5	8	1
1E4 * 5E5	32	2
1E4 * 10E5	80	3
5E4 * 1E5	22	1
5E4 * 5E5	44	4
5E4 * 10E5	66	4
1E5 * 1E5	34	2
1E5 * 5E5	68	2
1E5 * 10E5	94	3

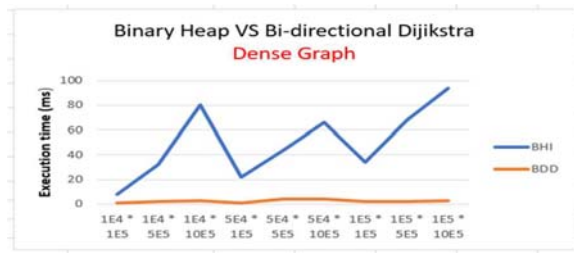


Fig. 17. Performance curve between BHI and BDD for Dense Graph

IV. CONCLUSION

After closely comparing the execution time of four algorithms on various real-world test cases, Bidirectional Dijkstra algorithm comes out to be the most efficient algorithm out of the four algorithms on the basis of time complexity. The chosen algorithm wins the race among all other algorithms in almost every test case. Therefore, it is proved that this algorithm can process huge real world graphs in a very small amount of time. The present paper also investigates the performances of various implementation approaches applied to Dijkstra algorithm for sparse and dense graphs. As a future scope of the present research, an intelligent system may be

developed in future to decide the applicability of a algorithm by seeing the nature of the problem in network oriented domains.

V. REFERENCES

- [1] R.K. Arjun, P. Reddy, Shama, and M. Yamuna, "Research on the optimization of Dijkstra's algorithm and its applications", *Int. J. Sci. Tech. Manag.*, vol. 4, no. 1, pp. 304 – 309, 2015.
- [2] T.H. Cormen, C.E. Leiserson, R.L. Rivest and C. Stein, "Dijkstra's algorithm". *Introduction to Algorithms*, 2nd ed., MIT Press and McGraw-Hill. pp. 595–601, 2001, ISBN 0-262-03293-7.
- [3] E.W Dijkstra, "A note on two problems in connection with graphs", *Numerische Mathematik*, vol. 1 , pp. 269–271, 1959.
- [4] T.B. de Silveira, E. M. Duque, S.J.F. Guimarães, H. T. Marques-Neto and H. C. de Freitas, "Proposal of Fibonacci heap in the Dijkstra algorithm for low-power ad-hoc mobile transmissions". *IEEE Latin America Transactions*, vol. 18, issue 3, pp. 623-630, 2020, doi: 10.1109/TLA.2020.9082735.
- [5] T. Porter and I. Simon, "Random insertion into a priority queue structure". *IEEE Transactions on Software Engineering*. SE-1 (3), 292–298, 1975, doi:10.1109/TSE.1975.6312854.
- [6] L. Chen, J. Zhou, J. Li and Y. Chen, "Bidirectional Dijkstra algorithm for best-routing of urban traffic network". *Proceedings SPIE*, vol. 6754, *Geoinformatics 2007: Geospatial Information Technology and Applications*, 675418, 2007, <https://doi.org/10.1117/12.764679>.